

Les Réseaux de Neurones Artificiels

Explication du cours — diapositives 1 à 92

Notes de lecture, définitions et intuitions

L'idée directrice à garder en tête tout du long : un réseau de neurones est une machine qui apprend, à partir d'exemples, une fonction qui transforme une entrée x en une sortie y . Tout le reste — perceptron, couches cachées, rétropropagation — n'est qu'une manière de construire cette fonction et de régler ses paramètres.

1. Introduction : d'où vient l'idée ?

Le neurone biologique

Le point de départ est une observation du cerveau : environ 10^{11} neurones, chacun connecté à 10^3 - 10^4 autres via des synapses. Un neurone reçoit des signaux par ses dendrites, les combine, et s'il est suffisamment stimulé, il « s'active » (potentiel d'action) et transmet un signal via son axone vers d'autres neurones.

Pourquoi ce modèle séduit : il apprend, il est robuste aux pannes (perdre quelques neurones ne casse pas tout), il tolère l'information incomplète, et il calcule de façon massivement parallèle.

L'idée du « neurone formel »

L'ambition est de prendre cette inspiration biologique et d'en faire un objet mathématique simple.

Quelques repères historiques :

- **McCulloch & Pitts (1943)** : premier modèle mathématique du neurone.
- **Hebb (1949)** : une règle d'apprentissage — « les neurones qui s'activent ensemble se lient ensemble » (fire together, wire together).
- **Rosenblatt (1958-1962)** : le Perceptron, le premier neurone qu'on sait effectivement entraîner.
- **Minsky & Papert (1969)** : démontrent les limites du perceptron, ce qui freine la recherche pendant près de 20 ans.
- **1985** : le perceptron multi-couches et la rétropropagation relancent le domaine.

Pourquoi s'intéresser aux réseaux de neurones ?

Attraits

- Calculs parallélisables
- Robustesse (le réseau est distribué sur de nombreuses petites unités)
- Algorithmes d'apprentissage simples
- Usage très général

Défauts

- Opacité des raisonnements : on ne comprend pas facilement pourquoi le réseau a pris telle décision (le problème d'explicabilité)

2. Le Perceptron : la brique de base

Définition

Un neurone artificiel reçoit un vecteur d'entrée $x = (x_1, \dots, x_n)$, et calcule une combinaison linéaire pondérée de ces entrées :

$$h(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n + w_0 = \sum_j w_jx_j + w_0$$

- w_j sont les poids (l'équivalent de la force des synapses).

- w_0 est le biais (un décalage constant, souvent représenté comme un poids attaché à une entrée fictive $x_0 = 1$).

Le résultat $h(x)$ passe ensuite dans une fonction pour décider de la sortie (au départ, simplement le signe).

Le perceptron comme séparateur linéaire — l'intuition géométrique

C'est le point le plus important à bien visualiser. L'équation

$$\sum_j w_j x_j + w_0 = 0$$

est l'équation d'un hyperplan (une droite en 2D, un plan en 3D, etc.). Ce plan coupe l'espace en deux régions : d'un côté $h(x) \geq 0$, de l'autre $h(x) < 0$. Le perceptron classe donc un point x en regardant simplement de quel côté de cette frontière il tombe.

Concrètement, avec deux classes de points (rouges et bleus) dispersés dans le plan, apprendre un perceptron revient à trouver la droite qui les sépare le mieux.

Comment apprend-on les poids ? L'intuition d'abord

L'idée de la règle du perceptron est très simple, presque du bon sens : on ne corrige les poids que quand on se trompe.

- Si l'exemple devrait être positif ($y = +1$) mais que $w \cdot x < 0$ (le perceptron le classe négatif), on augmente $w \cdot x$ pour cet exemple.
- Si l'exemple devrait être négatif ($y = -1$) mais que $w \cdot x > 0$, on diminue $w \cdot x$.

La règle de mise à jour est :

$$w' = w + \eta \cdot y_i \cdot x_i$$

où η (eta) est le taux d'apprentissage (un petit nombre qui contrôle la taille du pas de correction).

Géométriquement : on ajoute au vecteur poids w une petite fraction du vecteur x mal classé, ce qui fait pivoter la frontière de décision pour qu'elle se rapproche du bon côté.

Algorithme complet : tant que ce n'est pas convergé, on parcourt tous les exemples ; à chaque exemple mal classé ($y_i \cdot w \cdot x_i \leq 0$), on met à jour $w \leftarrow w + \eta y_i x_i$; on recommence jusqu'à ce qu'une passe complète ne produise plus aucune correction.

Reformulation en descente de gradient

On peut aussi voir l'apprentissage comme la minimisation d'une fonction d'erreur (le risque empirique) :

$$R_{Emp}(w) = - \sum (x_j \in \mathcal{M}) w^T x_j \cdot u_j$$

où $\mathcal{M}$ est l'ensemble des exemples mal classés. On veut que $w \cdot x$ soit du même signe que la classe attendue. Cette fonction se minimise par descente de gradient — l'idée générale (qu'on retrouvera partout dans la suite) : on calcule la pente (le gradient) de l'erreur par rapport aux poids, et on déplace les poids dans la direction qui fait diminuer l'erreur.

Une propriété remarquable

Théorème de convergence du perceptron : si les exemples sont linéairement séparables (il existe une droite/hyperplan qui les sépare parfaitement), alors l'algorithme du perceptron converge en un nombre fini d'étapes, quel que soit le nombre d'exemples, leur distribution, ou la dimension de l'espace d'entrée.

C'est une garantie forte — mais elle repose sur une hypothèse cruciale : que les données soient effectivement séparables linéairement.

La limite fatale : le XOR

Le perceptron ne peut apprendre que des séparations linéaires. Le problème classique qui l'illustre est le XOR (ou exclusif) :

Entrée x1	Entrée x2	Sortie XOR
0	0	0
0	1	1
1	0	1
1	1	0

Si vous placez ces 4 points dans le plan (x_1 , x_2), vous verrez qu'aucune droite unique ne peut séparer les points de sortie 1 des points de sortie 0 — contrairement à AND ou OR qui sont linéairement séparables.

À retenir : c'est exactement la démonstration de Minsky & Papert (1969) qui a mis un coup d'arrêt à la recherche sur les réseaux de neurones pendant près de 15 ans : un simple neurone ne « voit » pas la structure XOR.

3. Le Perceptron Multi-Couches (PMC / MLP)

L'idée pour dépasser la limite du perceptron simple

La solution au problème du XOR (et plus généralement à la limitation « séparation linéaire uniquement ») consiste à empiler plusieurs neurones en couches successives. C'est le Perceptron Multi-Couches.

Structure typique :

- une couche d'entrée (les x_k , pas vraiment des neurones, juste les données),
- une ou plusieurs couches cachées,
- une couche de sortie (produit y_k , comparée à la sortie désirée u_k).

Chaque flèche entre deux neurones porte un poids ; l'information circule de la gauche vers la droite (propagation « avant » ou feedforward).

La propagation avant (feedforward)

Pour chaque neurone k , on calcule :

$$a_k = \sum_j w_{jk} \cdot \phi_j \quad ; \quad y_k = g(a_k)$$

- a_k est l'activation (la somme pondérée des entrées de ce neurone, comme dans le perceptron simple).
- g est la fonction d'activation, qui transforme cette somme brute en une sortie. C'est l'ingrédient clé qui manquait au perceptron simple : en empilant des fonctions non linéaires, le réseau peut représenter des frontières de décision courbes, pas seulement des droites.

Les fonctions d'activation

- **Heaviside** (fonction à seuil) : 0 si $x < 0$, 1 sinon — la fonction historique, mais elle n'est pas dérivable, donc mal adaptée à la descente de gradient.
- **Sigmoïde** : $g(a) = 1 / (1 + e^{-a})$, sortie entre 0 et 1, dérivable partout, avec la propriété pratique $g'(a) = g(a)(1-g(a))$.
- **Tanh** : variante entre -1 et 1.
- **ReLU** : $\max(0, x)$, très utilisée aujourd'hui pour les réseaux profonds.

Pour la couche de sortie, le choix dépend de la tâche :

- **Classification** : sigmoïde (sortie interprétable comme une probabilité entre 0 et 1) ou softmax (pour plusieurs classes).
- **Régression** (prédire un nombre continu) : fonction linéaire en sortie.

Pourquoi le XOR devient soluble avec une couche cachée

Avec une couche cachée intermédiaire, chaque neurone caché peut apprendre sa propre droite de séparation partielle, et le neurone de sortie combine ces frontières partielles pour construire une frontière globale non linéaire, capable de « découper » le plan en zones complexes — suffisant pour résoudre XOR (le cours donne un exemple concret avec 9 poids qui résolvent XOR à l'aide de 2 neurones cachés + 1 neurone de sortie).

Le rôle profond de la couche cachée : le changement de représentation

C'est un point conceptuel essentiel, souvent sous-estimé. Chaque couche du réseau ne fait pas que « calculer » — elle transforme l'espace des données en une nouvelle représentation (des « variables latentes »). Une couche linéaire déforme l'espace de façon affine (rotation, étirement) ; une couche avec ReLU « écrase » une partie de l'espace vers zéro ; une couche avec normalisation L2 projette les points sur une sphère.

Idée clé : le but de cette succession de transformations est de rapprocher progressivement la distribution des sorties de la distribution cible souhaitée. Chaque couche successive doit rendre les classes de plus en plus séparables, jusqu'à ce que la dernière couche puisse les distinguer trivialement. Poussée à l'extrême (beaucoup de couches), cette idée donne naissance au deep learning : chaque couche apprend une abstraction un peu plus haut niveau que la précédente.

4. L'apprentissage dans les PMC : la rétropropagation du gradient

Le problème à résoudre

On dispose d'un échantillon d'apprentissage (paires entrée/sortie désirée). On veut trouver les poids w qui minimisent l'erreur moyenne entre la sortie produite par le réseau et la sortie désirée :

$$w^* = \operatorname{argmin}_w (1/m) \sum_l [y(x_l; w) - u(x_l)]^2$$

C'est une régression sur les poids : on cherche, parmi toutes les combinaisons de poids possibles, celle qui fait le moins d'erreur sur les exemples connus.

L'algorithme général : la descente de gradient

Le principe est itératif :

$$w^{(t)} = w^{(t-1)} - \eta \nabla_E w^{(t)}$$

Autrement dit : à chaque étape, on regarde dans quelle direction l'erreur augmente le plus (le gradient), et on va dans la direction opposée, d'un petit pas de taille η . On répète jusqu'à ce que l'erreur cesse de diminuer significativement.

Deux variantes :

- **Mode hors-ligne (batch)** : on calcule le gradient moyen sur tout l'ensemble d'apprentissage avant de mettre à jour les poids.
- **Mode en-ligne (stochastique)** : on met à jour les poids après chaque exemple individuellement. C'est plus bruité mais permet souvent d'échapper aux minima locaux.

Pourquoi « rétro-propagation » ? L'astuce du calcul

Le problème technique est le suivant : pour un neurone dans une couche cachée, comment savoir de combien il a contribué à l'erreur finale, alors que cette erreur n'est mesurée qu'à la toute fin du réseau (en sortie) ?

La réponse s'appuie sur la règle de dérivation en chaîne (chain rule). On définit, pour chaque neurone i , une quantité δ_i (le « signal d'erreur » local) :

- **Pour un neurone de sortie k** : $\delta_k = (y_k^l - \hat{y}_k^l) \cdot g'(a_k)$ — l'écart entre sortie désirée et sortie obtenue, pondéré par la sensibilité locale de la fonction d'activation.
- **Pour un neurone caché j** : $\delta_j = g'(a_j) \cdot \sum_k w_{jk} \cdot \delta_k$ — c'est-à-dire qu'on fait remonter (d'où « rétro-propagation ») les erreurs δ_k des neurones de la couche suivante, pondérées par les poids qui les relient à j .

Intuition en une phrase : un neurone caché est « responsable » de l'erreur finale proportionnellement à l'influence qu'il a eue sur les neurones qu'il alimente. On propage donc l'erreur de la sortie vers l'entrée, couche par couche, en sens inverse de la propagation normale — d'où le nom.

Une fois les δ calculés, la mise à jour des poids est simple :

$$\Delta w_{ij} = \eta(t) \cdot \delta_j \cdot y_i$$

(le produit du signal d'erreur du neurone en aval et de la sortie du neurone en amont — cohérent avec la règle de Hebb : on renforce les connexions qui ont contribué à corriger l'erreur).

L'algorithme complet, résumé en 7 étapes

1. Présenter un exemple de l'ensemble d'apprentissage.
2. Calculer l'état du réseau (propagation avant).
3. Calculer l'erreur (sortie obtenue vs désirée).
4. Calculer les gradients (rétropropagation).
5. Modifier les poids.
6. Vérifier le critère d'arrêt.
7. Recommencer.

Un aparté biologiquement intéressant

Le cours souligne que la rétropropagation, bien qu'extrêmement efficace, est peu plausible biologiquement (le cerveau n'a pas de mécanisme connu pour « renvoyer » une erreur globale couche par couche en sens inverse). À l'inverse, la règle de Hebb (« les neurones qui s'activent ensemble se lient ») est purement locale — chaque connexion se renforce en fonction de l'activité conjointe de ses deux neurones, sans coordination globale — donc plus plausible biologiquement, mais elle ne minimise pas explicitement une fonction de coût précise.

Les dangers pratiques de l'apprentissage

- **Minima locaux** : la fonction d'erreur, dans l'espace des poids, ressemble à un paysage montagneux plein de creux ; la descente de gradient peut se retrouver coincée dans un creux qui n'est pas le plus bas possible. L'apprentissage stochastique (en-ligne), plus bruité, aide parfois à s'en échapper.
- **Sur-apprentissage (overfitting)** : le réseau devient si bon sur les exemples d'apprentissage qu'il « apprend par cœur » leurs particularités, y compris le bruit, et généralise mal à de nouveaux exemples. On le voit très concrètement : l'erreur sur l'échantillon d'apprentissage continue de baisser, tandis que l'erreur sur un échantillon de validation, elle, se remet à augmenter après un moment.

La parade classique : l'arrêt précoce (early stopping) — on arrête l'apprentissage au moment où l'erreur de validation est minimale.

Obtenir une sortie interprétable comme une probabilité

Pour la classification, on veut souvent que la sortie du réseau soit interprétable comme « la probabilité que x appartienne à la classe C ». Pour cela, on utilise un signal d'erreur adapté, l'entropie croisée :

$$l(y_i, \hat{y}_i) = -[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

avec $\hat{y}_i$ produit par une sigmoïde. Cette fonction de coût, différente de l'erreur quadratique, « calibre » mieux la sortie comme une vraie probabilité.

5. Choisir la bonne architecture

Le compromis biais-variance

C'est LE concept central pour comprendre pourquoi le choix d'architecture (nombre de couches, nombre de neurones) est délicat :

- **Un réseau trop simple** (peu de neurones) → sous-apprentissage : il ne peut pas représenter la complexité réelle du phénomène (biais élevé), il généralise mal parce qu'il est structurellement trop rigide.
- **Un réseau trop complexe** (beaucoup de neurones/paramètres par rapport au nombre d'exemples) → sur-apprentissage : il colle trop aux données d'entraînement, y compris au bruit (variance élevée) — il « invente » des frontières trop découpées, trop fragmentées.
- **Le modèle « correct »** trouve l'équilibre : suffisamment de capacité pour capturer la vraie structure, pas trop pour ne pas mémoriser le bruit.

Formellement, l'erreur totale d'un modèle appris se décompose en :

- **Erreur d'approximation (biais)** : distance entre la meilleure fonction que la famille de modèles pourrait représenter et la vraie fonction cible.
- **Erreur d'estimation (variance)** : distance entre le modèle qu'on a effectivement appris (avec un échantillon limité et donc bruité) et le meilleur modèle possible dans cette famille.

À retenir : on ne peut généralement pas réduire les deux erreurs en même temps : augmenter la capacité du réseau réduit le biais mais augmente la variance, et vice versa.

Le codage de la couche de sortie

Pour un problème à c classes, deux approches :

- **Un seul neurone de sortie**, dont la valeur code la classe (moins courant pour $c > 2$).
- **Plusieurs neurones de sortie** (souvent un par classe, « one-hot encoding ») : la sortie est interprétée comme un vecteur de probabilités d'appartenance à chaque classe.

Les codes correcteurs d'erreur (ECOC)

Une astuce plus subtile pour la classification multi-classes : au lieu d'utiliser exactement un neurone de sortie par classe, on utilise plus de neurones que de classes (par exemple 10 neurones binaires pour coder 26 lettres au lieu de 5 qui suffiraient en théorie). Chaque classe est associée à un « mot code » — une suite de bits — et on choisit les codes pour qu'ils soient les plus différents possibles les uns des autres (en distance de Hamming).

Avantage : si un des 10 classifieurs binaires se trompe légèrement, on peut quand même retrouver la bonne classe en cherchant, parmi les codes connus, celui qui est le plus proche voisin de la sortie obtenue — un peu comme un code correcteur d'erreur en télécommunication. C'est une façon d'ajouter de la robustesse au bruit dans les prédictions.

En résumé

Pourquoi les réseaux de neurones sont-ils séduisants ?

1. **Grande capacité expressive** : ce sont, à la limite, des approximateurs universels (ils peuvent en théorie représenter n'importe quelle fonction raisonnable) — ce qui pose en creux la question de savoir comment ils arrivent à généraliser malgré cette énorme flexibilité (le problème d'induction).
2. **Les paramètres s'apprennent par descente de gradient** — une méthode générique, qui ne demande pas de concevoir à la main les règles de décision.
3. **Les calculs sont parallélisables** — chaque neurone d'une même couche peut être calculé indépendamment.
4. **On peut structurer l'architecture** pour refléter la structure du problème (par exemple avec les réseaux convolutifs pour les images).
5. **Ils apprennent des abstractions** — les couches successives construisent des représentations de plus en plus utiles du problème, sans qu'on ait besoin de les spécifier à la main.

Document préparé à partir du cours « Les réseaux de neurones artificiels » (A. Cornuéjols, AgroParisTech – INRAE MIA Paris-Saclay).